

EXAMEN DE DISEÑO Y ANÁLISIS DE ALGORITMOS

OSWALDO JOAQUIN ZAMORA RODRIGUEZ

SEMESTRE A26

ABRIL 2026

1 Respuestas

1.1 Demuestre que $256x^2 + 12x^3 + 12x + 8$ pertenece a $\Theta(x^3)$.

- $256x^2 + 12x^3 + 12x + 8 = \Theta(x^3)$

Por propiedad de Θ :

$$f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$$

Por definición de O :

$$O(g(n)) = \{f(n) : \exists(c > 0, n_0), 0 \leq f(n) \leq cg(n), \forall n \geq n_0\}$$

1. x^3 acota si se cumple que $\exists(c > 0, x_0) \wedge (0 \leq f(x) \leq x^3)$.
2. Supongamos $0 \leq 256x^2 + 12x^3 + 12x + 8 \leq cx^3$.
3.
$$0 \leq \frac{256}{x} + 12 + \frac{12}{x^2} + \frac{8}{x^3} \leq c$$
4. si $x = 1$ entonces $0 \leq 256 + 12 + 12 + 8 \leq c$ por lo que c sí puede existir.
5. entonces para todo $x > 1$ notemos que $\frac{256}{x} + 12 + \frac{12}{x^2} + \frac{8}{x^3}$ decrementa.
6. $\therefore \exists(c > 0, n_0) : 256 + 12 + 12 + 8 \leq c \wedge x_0 \geq 1$.
- 7.

$$\Rightarrow 256x^2 + 12x^3 + 12x + 8 = O(x^3)$$

Por definición de Ω :

$$\Omega(g(n)) = \{f(n) : \exists(c > 0, n_0), 0 \leq cg(n) \leq f(n), \forall n \geq n_0\}$$

1. x^3 acota si se cumple que $\exists(c > 0, x_0) \wedge (f(x) \geq cx^3 \geq 0)$.
2. Supongamos $256x^2 + 12x^3 + 12x + 8 \geq cx^3 \geq 0$.
3. sea $c \leq 12$ por los términos de la misma potencia.
4. si $x = 1$ entonces $256 + 12 + 12 + 8 \geq c \geq 0$ por lo que c sí puede existir.
5. entonces para todo $x > 1$ notemos que $256x^2 + 12x^3 + 12x + 8$ incrementa.
6. $\therefore \exists(c > 0, n_0) : c \leq 12 \wedge x_0 \geq 1$.
- 7.

$$\Rightarrow 256x^2 + 12x^3 + 12x + 8 = \Omega(x^3)$$

Como $256x^2 + 12x^3 + 12x + 8 = O(x^3) \wedge 256x^2 + 12x^3 + 12x + 8 = \Omega(x^3)$ podemos decir que:

$$256x^2 + 12x^3 + 12x + 8 = \Theta(x^3)$$

1.2 Dado un arreglo A que tiene n enteros A1, A2, . . . , An, generar una matriz B de n por n en donde Bij (para i ≤ j) contiene la suma de los números del arreglo A, desde Ai hasta Aj (Ai + Ai+1 + . . . + Aj). Los valores de Bij = 0 para i > j. Un algoritmo simple para resolver el problema seria como el dado en el Algoritmo 1.

- a) Dar un limite de la forma $O(f(n))$ para el tiempo de ejecucion del Algoritmo 1 y para una entrada de n elementos. (10 puntos)

Sea $f(n)$ la función que expresa la complejidad del Algoritmo 1.

1. Se procesa todo $i \in [1, n]$ de acuerdo con "for $i = 1, 2, \dots, n$ do".
2. Se procesa todo $j \in [i, n]$ para cada $i \in [1, n]$ de acuerdo con "for $j = i, i + 1, \dots, n$ do".
3. La suma toma un tiempo constante, pero se realiza $\forall A_{ij}$.
4. de acuerdo con (1.) la cantidad de iteraciones es congruente con $|[1, n]| = n$.
5. de acuerdo con (4. y 2.) la cantidad de iteraciones es congruente con

$$|[1, n] \times [i, n]| = |[1, n]| * |[i, n]| = n^2$$

6. En el peor de los casos se realiza la iteración desde $i = 0$ $j = n$

$$|[1, n] \times [i, n] \times [i, j]| = |[1, n]| * |[i, n]| * |[i, j]| = n^3$$

- 7.

$$\Rightarrow f(n) = O(n^3)$$

- b) Para la misma función f, demuestre que el tiempo de ejecución del algoritmo es también $\Omega(f(n))$ y que también es $\theta(f(n))$. (15 puntos)

Como siempre el recorrido se da desde $[1, n]$ sin una variación o excepción de acuerdo al valor de A_{ij} , entonces el mejor de los casos es también el peor de los casos:

1. En el mejor de los casos $i = j$ por lo que en ese caso $f(n) = 1$.
2. En el recorrido siempre se pasa por el peor, así como el mejor de los casos $f(n) = 1 + \dots + n^3$ de acuerdo con (a) 6. y b) 1.)
- 3.

$$\Rightarrow f(n) = \Omega(n^3)$$

Por propiedad de Θ :

$$f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$$

4. por b)3. y a)7. :

$$\therefore f(n) = \Theta(n^3)$$

- c) Dar un algoritmo alternativo para resolver el problema, que tenga un mejor tiempo de ejecución $O(g(n))$, donde $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$. Demuéstrelo. (25 puntos)

Como $B_{ij} = \sum_{k=i}^j A_k$ entonces $B_{ij} = A_i + \sum_{k=i+1}^j A_k = A_i + B_{i+1,j}$

Algoritmo...

- I. $\forall i = n, n-1, \dots, 2, 1 \{ //$ recorrido descendente.
- II. $\forall j = n, n-1, \dots, i \{ //$ recorrido descendente.
- III. $B_{ij} = \begin{cases} 0, \text{ si } i > j, \\ A_{ij}, \text{ si } i = j, \\ A_{ij} + B_{i+1,j}, \text{ si } i < j. \end{cases}$
- IV. $\}$
- V. $\}$

Demostración de $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$, sea $f(n)$ la función del algoritmo anterior:

1. Por I. se recorre $i \in [1, n] \Rightarrow g(n) = O(n)$
2. Por II. y 1. se recorre $j \in [i, n] \forall i \in [1, n] \Rightarrow g(n) = O(n^2)$
3. por a)7. y 2.:

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \lim_{n \rightarrow \infty} \frac{n^2}{n^3}$$

4.

$$\lim_{n \rightarrow \infty} \frac{1}{n} = \frac{1}{\infty} = 0$$

1.3 Dado el grafo de la Figura 1, demuestre que:

Sea T el MST donde por definición contiene la trayectoria S más corta, es decir $\forall e \in S; \sum_e l_e$ tiene el valor más bajo con respecto a otras.

Sean también las trayectorias $A = \{(a, b)\}$ y $B = \{(b, c), (c, a)\}$ que van de del nodo 'a' a 'b'.

- a) la arista (a, b) no puede estar en el MST. (10 puntos)

1. Supongamos que $(a, b) \in S$
2. $\sum_A l_a < \sum_B l_b$
3. Sustituyendo valores: $4 < 2 + 1$
4. Realizando la suma: $4 < 3$!!! Contradicción.
5. $\therefore (a, b) \notin S$

- b) la arista (c, a) debe estar en el MST. (10 puntos)

Considerando que el MST pasa por todos los nodos conectados, entonces pasa del nodo 'a' y el nodo 'b'...

1. $A \subset S \vee B \subset S$
2. por a) 5. $(a, b) \notin S$
3. Modus Tollendo Ponens considerando 1. y 2.

$$B \subset S$$

4. como $(c, a) \in B \Rightarrow (c, a) \in S$

1.5 Determine si el grafo de la Figura 3 es bipartita. Fundamente su respuesta. (20 puntos)

Un Grafo G es bipartito si $\exists V_1$ y $\exists V_2$ tal que generan una partición de V , así como $\forall (u, v) \in E$ se cumple que $u \in V_1$ y $v \in V_2$.

Demostración...

1. Podemos crear las particiones de nodos que no se conectan por una arista:

$$V_1 = \{h, f, d, b\}$$

$$V_2 = \{g, e, c\}$$

2. Pero como $\exists (a, d) \in E \wedge \exists (g, a) \in E \Rightarrow a \notin V_1 \wedge a \notin V_2$
3. $\therefore$ No se puede crear una partición que contenga al nodo 'a', en consecuencia G no es bipartito.

1.4 Calcule y demuestre el límite asintótico $O(f(n))$ para el segmento de código de la Figura 2.

El segmento de código tiene complejidad asintótica:

$$O(n \log n)$$

Inicialmente se tiene:

$$i = 2n$$

El ciclo **while** se ejecuta mientras $i > 2$ y en cada iteración el valor de i se divide entre 5:

$$i = \frac{i}{5}$$

Después de t iteraciones:

$$i = \frac{2n}{5^t}$$

El ciclo termina cuando:

$$\frac{2n}{5^t} \leq 2$$

Entonces:

$$n \leq 5^t$$

Aplicando:

$$t \geq \log_5(n)$$

Por lo tanto, el ciclo **while** se ejecuta:

$$O(\log n)$$

Dentro del ciclo **while**:

1. La instrucción `printf("1");` tiene costo $O(1)$.
2. El ciclo:

`for(j = 0; j < n/2; j++)`

se ejecuta aproximadamente $n/2$ veces, por lo que:

$$O(n)$$

El siguiente bloque:

`for(k = n; k > 1; k-)`

se ejecuta aproximadamente n veces, y contiene el ciclo:

`for(m = 1; m < 10; m++)`

el cual se ejecuta 9 veces (constante), por lo que:

$$n \cdot 9 = 9n \Rightarrow O(n)$$

Entonces, el costo total dentro de cada iteración del **while**

es:

$$O(1) + O(n) + O(n) = O(n)$$

Finalmente, como el ciclo **while** se ejecuta $O(\log n)$ veces:

$$O(n) \cdot O(\log n) = O(n \log n)$$

$$O(n \log n)$$